

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

Name: K. Dedeepya

Reg no: 192324252

1. A perceptron is trained to classify farmland soil as 'fertile' or 'degraded' using nitrogen, phosphorus, and potassium content as inputs. Derive the perceptron learning rule and explain convergence conditions. (SDG 2 – Zero Hunger)

1. Problem Statement

Farmland soil quality is important for crop production. Soil can be classified as fertile or degraded based on its nutrient content. A perceptron can be used to classify soil using Nitrogen (N), Phosphorus (P), and Potassium (K) values.

2. Sustainable Development Goal

SDG 2 – Zero Hunger

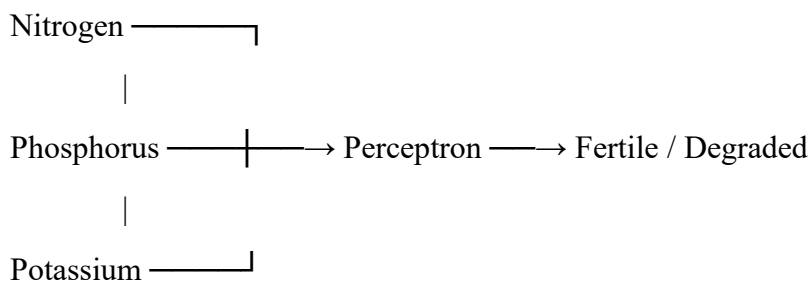
Identifying fertile soil helps farmers select suitable land for cultivation and improve agricultural productivity.

3. Learning Problem

This is a binary classification problem.

- **Inputs:** Nitrogen, Phosphorus and Potassium
- **Output:** Fertile / Degraded
- **Algorithm:** Single-Layer Perceptron
- **Learning Type:** Supervised Learning

4. Perceptron Representation



The perceptron calculates a weighted sum:

$$\text{Net input} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

where:

- x_1 = Nitrogen
- x_2 = Phosphorus
- x_3 = Potassium
- w_1, w_2, w_3 = weights
- b = bias

5. Activation Function

A step function is used:

Output = 1, if Net input ≥ 0

Output = 0, if Net input < 0

Here:

- **1 = Fertile**
- **0 = Degraded**

6. Perceptron Learning Rule

The weight update rule is: $w_i(\text{new}) = w_i(\text{old}) + \eta (t - y) x_i$

The bias is updated as: $b(\text{new}) = b(\text{old}) + \eta (t - y)$

where:

- η = learning rate
- t = target output
- y = predicted output
- x_i = input feature

7. Example

Assume:

- $N = 5$
- $P = 3$
- $K = 4$
- Target = 1
- Learning rate = 0.1
- Initial weights = 0
- Initial bias = 0

The perceptron calculates the weighted sum and produces a prediction.

If the prediction is incorrect, the weights are updated using: $\mathbf{w}_i(\text{new}) = \mathbf{w}_i(\text{old}) + \eta (t - y) \mathbf{x}_i$

For a target of 1 and prediction of 0: **Error** = $1 - 0 = 1$

Therefore: $w_1 = 0 + 0.1(1)(5) = 0.5$

$w_2 = 0 + 0.1(1)(3) = 0.3$

$w_3 = 0 + 0.1(1)(4) = 0.4$

$b = 0 + 0.1(1) = 0.1$

Updated weights are: **w = (0.5, 0.3, 0.4)**

8. Convergence Conditions

The perceptron learning algorithm converges when the training data is linearly separable.

This means a single straight decision boundary can separate fertile soil from degraded soil.

If the data is not linearly separable, the perceptron may continue updating its weights without reaching a stable solution.

9. Advantages

- Simple and easy to implement.
- Fast for binary classification.
- Easy to understand.
- Suitable for linearly separable data.

10. Limitations

- Cannot solve non-linear classification problems.
- Performance depends on the quality of soil data.
- Requires suitable training examples.
- Sensitive to feature scaling when feature ranges differ greatly.

11. Practical Application

The system can support:

- Soil quality monitoring
- Agricultural decision-making
- Fertilizer planning
- Crop selection
- Precision agriculture

12. Conclusion

A single-layer perceptron can classify farmland soil as fertile or degraded using Nitrogen, Phosphorus and Potassium values. The perceptron learning rule updates the weights based on prediction error. When the data is linearly separable, the algorithm converges to a suitable classification boundary. This application supports SDG 2 – Zero Hunger.

2. Using the perceptron algorithm, design a binary classifier that predicts whether a patient is at risk of malnutrition based on BMI and dietary intake. Show two iterations of weight updates with sample data. (SDG 2 – Zero Hunger)

1. Problem Statement

Malnutrition can affect a person's health and development. Early identification of people at risk can help provide appropriate nutritional support. A perceptron can classify a patient as at risk or not at-risk using BMI and dietary intake.

2. Sustainable Development Goal

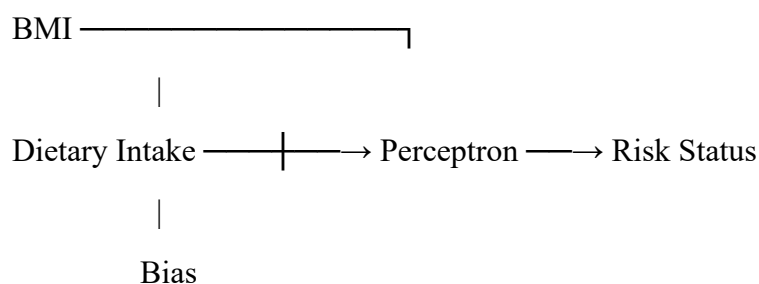
SDG 2 – Zero Hunger

Early identification of malnutrition supports better nutrition and helps reduce hunger-related health problems.

3. Learning Problem

- **Inputs:** BMI and dietary intake
- **Output:** At Risk / Not at Risk
- **Algorithm:** Single-Layer Perceptron
- **Learning Type:** Supervised Learning

4. Perceptron Model



The weighted sum is: $\text{Net} = w_1 x_1 + w_2 x_2 + b$

5. Activation Function

If $\text{Net} \geq 0 \rightarrow 1$

If $\text{Net} < 0 \rightarrow 0$

- 1 = At Risk
- 0 = Not at Risk

6. Weight Update Rule

$$w_i(\text{new}) = w_i(\text{old}) + \eta (t - y) x_i$$

$$b(\text{new}) = b(\text{old}) + \eta (t - y)$$

7. Sample Data

BMI	Dietary Intake	Target
16	30	1
22	70	0
17	35	1
24	80	0

Here, 1 represents At Risk and 0 represents Not at Risk.

8. Iteration 1

Assume:

- BMI = 16
- Dietary Intake = 30
- Target = 1
- Initial weights = 0
- Bias = 0
- Learning rate = 0.1

$$\text{Weighted sum: Net} = (0)(16) + (0)(30) + 0 = 0$$

Assume the initial prediction is: $y = 0$

$$\text{Error: } t - y = 1 - 0 = 1$$

Weight updates:

$$w_1 = 0 + 0.1(1)(16) = 1.6$$

$$w_2 = 0 + 0.1(1)(30) = 3.0$$

$$b = 0 + 0.1(1) = 0.1$$

Updated weights: $w_1 = 1.6$, $w_2 = 3.0$, $b = 0.1$

9. Iteration 2

Consider another sample:

- BMI = 22
- Dietary Intake = 70
- Target = 0

Using the updated weights:

$$\text{Net} = (1.6)(22) + (3.0)(70) + 0.1$$

$$\text{Net} = 245.3$$

The prediction is: $y = 1$

Target: $t = 0$

$$\text{Error: } t - y = 0 - 1 = -1$$

Update:

$$w_1 = 1.6 + 0.1(-1)(22) = -0.6$$

$$w_2 = 3.0 + 0.1(-1)(70) = -4.0$$

$$b = 0.1 + 0.1(-1) = 0$$

Therefore, after the second iteration:

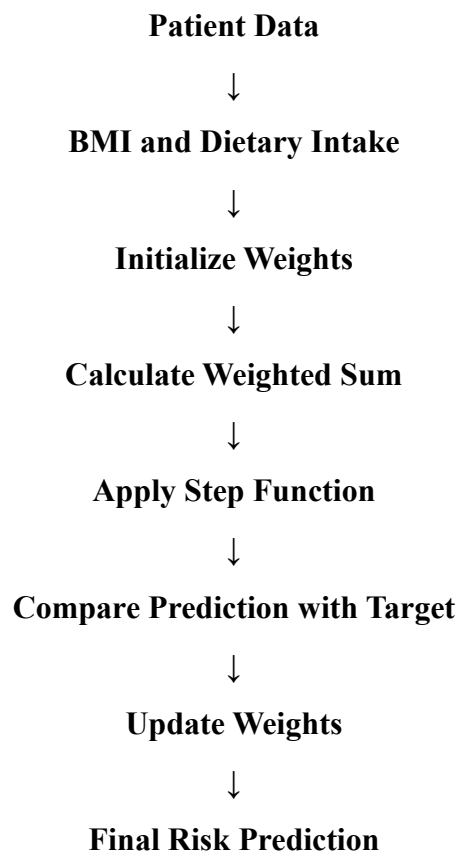
$$w_1 = -0.6$$

$$w_2 = -4.0$$

$$b = 0$$

This demonstrates how the perceptron changes its weights when its prediction is incorrect.

10. Workflow



11. Advantages

- Simple classification method.
- Fast training.
- Easy to understand.
- Useful for binary risk classification.

12. Limitations

- Cannot model complex non-linear relationships.
- Real malnutrition prediction requires more health and nutritional factors.
- Poor-quality data can affect predictions.
- It should support, not replace, professional medical assessment.

13. Conclusion

A perceptron can classify malnutrition risk using BMI and dietary intake. The weight update rule allows the model to learn from classification errors. Repeated training can improve the classification of suitable datasets. This application supports SDG 2 – Zero Hunger.

3. Explain the vanishing gradient problem in backpropagation and its impact on a deep network trained to detect early signs of crop disease from leaf images. (SDG 2 – Zero Hunger)

1. Problem Statement

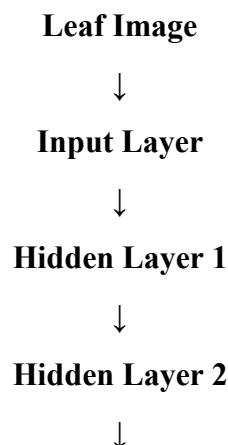
Deep Neural Networks can be used to detect crop diseases from leaf images. However, during backpropagation, the gradients passed from the output layer toward earlier layers can become extremely small. This problem is called the vanishing gradient problem.

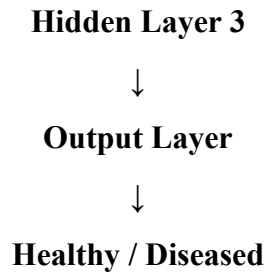
2. Sustainable Development Goal

SDG 2 – Zero Hunger

Early crop disease detection can reduce crop losses and support food production.

3. Neural Network Representation





4. What is a Gradient?

A gradient indicates how much a neural network's error changes when a weight changes.

During backpropagation, gradients are used to update the weights.

5. What is the Vanishing Gradient Problem?

In a deep network, the gradient is repeatedly multiplied while moving backward through many layers.

If the gradient values become very small: $0.1 \times 0.1 \times 0.1 \times 0.1 = 0.0001$

The early layers receive almost no useful update.

As a result, these layers learn very slowly.

6. Effect on Crop Disease Detection

For crop disease detection:

- The output layer identifies healthy or diseased leaves.
- Errors are propagated backward.
- Very small gradients may reach the first layers.
- Early layers learn important image features very slowly.
- Training becomes inefficient.
- The model may have difficulty learning useful patterns.

7. Main Causes

The problem commonly occurs when:

- The network is very deep.
- Sigmoid or tanh activation functions are used extensively.
- Gradients are repeatedly multiplied by values smaller than 1.
- Poor initialization can make the problem worse.

8. Example

Suppose the gradient at the output is: **0.5**

After passing through several layers:

0.5
↓
0.05
↓
0.005
↓
0.0005
↓
0.00005

The gradient becomes extremely small.

Therefore, the early layers receive almost no learning signal.

9. Solutions

1. ReLU Activation: ReLU is commonly used because it reduces the vanishing gradient problem compared with sigmoid in many deep networks.

2. Proper Weight Initialization: Suitable initialization methods help maintain useful gradient values.

3. Batch Normalization: Batch normalization can make training more stable.

4. Residual Connections: Residual connections allow information and gradients to pass through deeper networks more effectively.

10. Advantages of Addressing the Problem

- Faster training.
- Better feature learning.
- Improved disease classification.
- More stable deep network training.

11. Conclusion

The vanishing gradient problem occurs when gradients become extremely small during backpropagation. In deep crop disease detection networks, this prevents early layers from learning useful image features efficiently. Techniques such as ReLU, proper initialization, batch normalization and residual connections can reduce the problem.

4. Discuss how the learning rate parameter in backpropagation affects convergence speed and accuracy when training a model to predict renewable energy output from weather forecasts. (SDG 7 – Affordable and Clean Energy)

1. Problem Statement

Renewable energy output depends on factors such as sunlight, wind speed, temperature and weather conditions. A neural network can predict renewable energy output from these variables. During training, the learning rate controls how much the network changes its weights after each update.

2. Sustainable Development Goal

SDG 7 – Affordable and Clean Energy

Accurate renewable energy prediction supports efficient use and management of clean energy resources.

3. Learning Rate

The learning rate is represented by: η (eta)

It determines the size of the weight update during backpropagation.

The general update rule is: **New Weight = Old Weight – $\eta \times \text{Gradient}$**

4. Small Learning Rate

When η is very small:

Small Learning Rate



Small Weight Updates



Slow Learning



Slow Convergence

Advantages

- More stable learning.
- Less chance of overshooting the minimum.

Disadvantage

- Training can take a very long time.

5. Large Learning Rate

When η is very large:

Large Learning Rate



Large Weight Updates



Fast Movement



Possible Overshooting

The model may jump over the best solution and fail to converge properly.

6. Suitable Learning Rate

A suitable learning rate provides a balance between speed and stability.

Learning Rate



Weight Update



Model Training



Error Reduction



Convergence

7. Comparison

Learning Rate	Training Speed	Stability	Problem
Very Small	Slow	High	Slow convergence
Moderate	Good	Good	Usually, suitable
Very Large	Very Fast	Low	May not converge

8. Example

Suppose the current weight is: $w = 0.5$

and the gradient is:

Gradient = 0.2

Learning rate = 0.1

New weight = $0.5 - (0.1 \times 0.2)$

New weight = 0.48

Learning rate = 0.01

New weight = $0.5 - (0.01 \times 0.2)$

New weight = 0.498

The smaller learning rate produces a smaller update.

9. Effect on Renewable Energy Prediction

For renewable energy prediction, an appropriate learning rate helps the neural network learn relationships between:

- Solar radiation
- Wind speed
- Temperature
- Humidity
- Cloud conditions
- Historical energy output

A very small learning rate can make training slow, while a very large learning rate can make prediction accuracy unstable.

10. Recommendations

- Start with a moderate learning rate.
- Monitor training error.
- Reduce the learning rate when training becomes unstable.
- Use learning-rate scheduling when required.

11. Advantages

- Controls training speed.
- Helps control convergence.
- Affects final model performance.
- Important for stable neural network training.

12. Limitations

- The best learning rate depends on the dataset and model.
- A single fixed learning rate may not work throughout training.
- Very large values can cause divergence.

13. Conclusion

The learning rate is an important parameter in backpropagation. A small learning rate provides stable but slow learning, while a large learning rate can cause unstable training. A suitable learning rate provides a balance between convergence speed and accuracy, which is important for renewable energy output prediction.

5. Compare batch, stochastic, and mini-batch gradient descent in the context of training a backpropagation network on large-scale climate sensor data for extreme weather prediction. (SDG 13 – Climate Action)

1. Problem Statement

Climate monitoring systems generate large amounts of sensor data from temperature, rainfall, humidity, wind speed and other environmental variables. Neural networks can use this data to predict extreme weather events. Different gradient descent methods can be used to train the neural network.

2. Sustainable Development Goal

SDG 13 – Climate Action

Accurate prediction of extreme weather can support disaster preparedness and climate risk management.

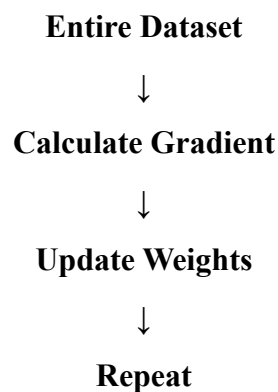
3. Gradient Descent

Gradient descent is an optimization technique used to reduce the error of a Machine Learning model by adjusting its weights.

The general update rule is: $w(\text{new}) = w(\text{old}) - \eta \times \text{gradient}$

4. Batch Gradient Descent

Batch Gradient Descent uses the entire training dataset to calculate each weight update.



Advantages

- Stable updates.
- Accurate gradient estimation.
- Suitable for smaller datasets.

Limitations

- Slow for very large datasets.
- Requires more memory.
- Each update can take considerable time.

5. Stochastic Gradient Descent

Stochastic Gradient Descent uses one training example at a time.

One Data Sample



Calculate Gradient



Update Weights



Next Sample

Advantages

- Faster individual updates.
- Requires less memory.
- Can work well with very large datasets.

Limitations

- Updates are noisy.
- Training may fluctuate.
- May not converge smoothly.

6. Mini-Batch Gradient Descent

Mini-Batch Gradient Descent divides the dataset into small batches and updates the weights after each batch.

Large Dataset



Small Batch



Calculate Gradient



Update Weights



Next Batch

Advantages

- Faster than batch training.
- More stable than stochastic training.
- Efficient use of memory.

- Well suited to large datasets.

7. Comparison

Feature	Batch	Stochastic	Mini-Batch
Data per update	Entire dataset	One sample	Small batch
Speed	Slow	Fast updates	Fast
Stability	High	Low	Good
Memory	High	Low	Moderate
Large dataset	Less suitable	Suitable	Highly suitable
Training behaviour	Smooth	Noisy	Relatively smooth

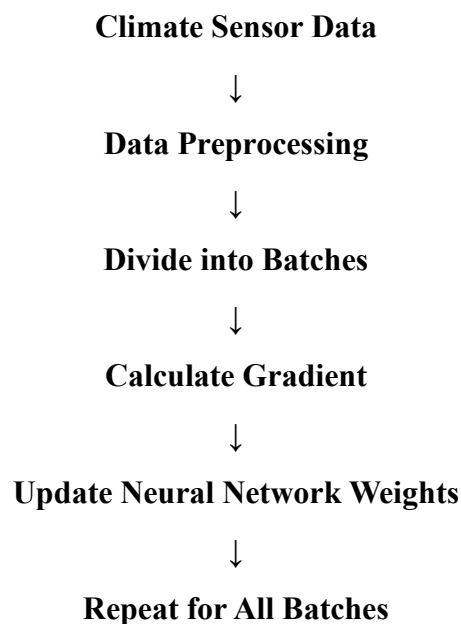
8. Application to Climate Sensor Data

Climate sensor datasets may contain thousands or millions of records. Each record can include:

- Temperature
- Rainfall
- Humidity
- Wind speed
- Atmospheric pressure
- Historical weather conditions

Mini-Batch Gradient Descent is often a practical choice because it provides a balance between computational efficiency and stable learning.

9. Simple Training Workflow





Extreme Weather Prediction

10. Advantages

- Supports large climate datasets.
- Improves training efficiency.
- Can process data in manageable batches.
- Useful for neural network training.

11. Limitations

- Batch Gradient Descent requires high memory.
- Stochastic Gradient Descent can be unstable.
- Mini-Batch requires selecting an appropriate batch size.

12. Conclusion

Batch, Stochastic and Mini-Batch Gradient Descent differ mainly in the amount of data used for each weight update. Batch provides stable updates, Stochastic provides frequent but noisy updates, and Mini-Batch provides a balance between the two. For large-scale climate sensor data, Mini-Batch Gradient Descent is generally a suitable choice because it provides good efficiency and stable training. This supports SDG 13 – Climate Action.